ProEm: A Beginner's Guide to the Toolchain

YAML, simulation, Icarus Verilog, cocotb, the ASIC flow, processes and Tiny Tapeout

Paul Vincent Bezzina

28 September 2026

Abstract

This guide explains, from the beginning, every tool and file that takes ProEm from SystemVerilog to a chip. It assumes you know Verilog from Quartus and ModelSim, but not the open-source ASIC world. Each section explains one idea in general terms, then shows exactly where it appears in the ProEm repository. The last sections are a checklist of what to do next and a glossary.

Contents

1 The big picture **3**

2 The files in your repository **3**

3 YAML **4**
- 3.1 What YAML is . . . 4
- 3.2 Your `info.yaml` . . . 4
- 3.3 Workflow files are YAML too . . . 4

4 JSON and `src/config.json` **5**

5 Simulation **7**
- 5.1 What a simulator does . . . 7
- 5.2 Icarus Verilog . . . 7
- 5.3 Waveforms . . . 7
- 5.4 RTL and gate-level simulation . . . 7
- 5.5 Your C++ test suite (CXXRTL) . . . 7

6 cocotb **8**
- 6.1 What cocotb is . . . 8
- 6.2 The pieces in `test/` . . . 8
- 6.3 Reading a test . . . 8
- 6.4 Running it yourself (Windows) . . . 8

7 Hands-on: simulating ProEm yourself **9**
- 7.1 Which one to use . . . 9
- 7.2 One-time setup (Windows, with WSL) . . . 9
- 7.3 cocotb, step by step . . . 9
- 7.4 C++ (CXXRTL), step by step . . . 11

8 The ASIC flow (LibreLane) **13**
- 8.1 1. Lint and synthesis (Verilator, Yosys) . . . 13
- 8.2 2. Floorplan and power grid . . . 13

8.3 3. Placement (OpenROAD) . . . 13
8.4 4. Clock tree synthesis (CTS) . . . 13
8.5 5. Resizing and timing repair . . . 13
8.6 6. Routing . . . 14
8.7 7. Antenna repair, fill and extraction . . . 14
8.8 8. Sign-off . . . 14
8.9 9. Output . . . 14

9 Timing, explained simply **14**
9.1 Setup and hold . . . 14
9.2 Corners . . . 15
9.3 Slew . . . 15

10 Processes and PDKs **15**
10.1 What a process is . . . 15
10.2 The processes you have met . . . 15

11 How Tiny Tapeout works **15**
11.1 Sharing one chip . . . 15
11.2 Your pins . . . 16
11.3 Templates . . . 16

12 GitHub Actions: what happens when you push **16**
12.1 Workflows . . . 16
12.2 ProEm's workflows . . . 17
12.3 Reading the results . . . 17

13 Git in five commands **17**

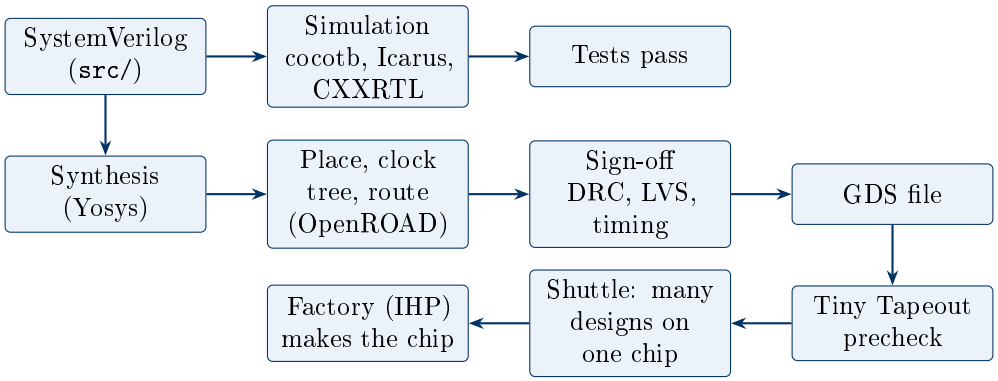
14 What to do next **17**

15 Glossary **18**

1 The big picture

An FPGA is a chip that already exists; Quartus configures it. An ASIC is a chip that does not exist yet: the tools must decide where every transistor goes and draw the photographic masks a factory uses to make it. The output of that process is a **GDS file**, a drawing of every layer of the chip.

The journey for ProEm has two branches that meet at the end: one checks the design works (simulation), the other turns it into a layout (the ASIC flow).



For you, almost all of this runs automatically: when you push to GitHub, **GitHub Actions** (computers GitHub lends you) run the tests and the whole ASIC flow, and show a green tick or a red cross. Your job is to write the SystemVerilog, keep the tests passing, and make sure the `gds` run is green. Jane Street's email says exactly this: start from the CMOS5L template, let the Tiny Tapeout Actions build the GDS and run the precheck, and submit a design with a passing build.

2 The files in your repository

File or folder	What it is for
`src/`	Your SystemVerilog. `top/tt_um_vertrex_star_proem.sv` is the Tiny Tapeout wrapper; everything else sits under it.
`src/config.json`	Settings for the ASIC flow (LibreLane): clock period, placement density, timing margins. Section 4.
`info.yaml`	The Tiny Tapeout project description: name, top module, tile size, source files, pin names. Section 3.
`test/`	The cocotb test: `test.py` (Python), `tb.v` (a Verilog wrapper), `Makefile`, `requirements.txt`. Section 6.
`sim/`, `scripts/run.sh`	The larger C++ test suite (CXXRTL): 13 pin-level tests plus random programs. Section 5.5.
`.github/workflows/`	The recipes GitHub Actions follow on every push: `gds.yaml`, `test.yaml`, `sim.yaml`, `docs.yaml`. Section 12.
`docs/`	The ISA and design documents, the datasheet page `info.md`, diagrams.
`gds/`	A copy of the finished layout (GDS, LEF), images, and a results summary.
`host/`	The Python host library (`proem_host.py`) that builds instruction bytes and decodes replies, and a README on how to drive the chip.
`formal/`	Formal proofs: harnesses and `.sby` files for SymbiYosys (see the separate formal verification guide).

3 YAML

3.1 What YAML is

YAML is a plain-text format for writing down structured settings that both people and programs can read. It has only three building blocks:

- **A value**: text or a number, such as `50000000` or `"6x4"`.
- **A map** (also called a dictionary): names paired with values, written `name: value`.
- **A list**: items written one per line, each starting with a dash `-`.

Nesting is shown by **indentation**, like Python. Everything indented under a line belongs to it.

```
project:              # a map called "project"...
  title: "ProEm"      # ...containing a value called "title"
  tiles: "6x4"
  source_files:       # ...and a list called "source_files"
    - "core/line_codec.sv"
    - "top/protocol_emulator.sv"
```

The rules that catch beginners:

- Indent with **spaces only**, never tabs. Two spaces per level is the convention.
- A colon must be followed by a space: `tiles: "6x4"`, not `tiles:"6x4"`.
- `#` starts a comment; the rest of the line is ignored.
- Put text in quotes when it contains a colon, a `#`, or starts with a special character. Quoting all text, as your `info.yaml` does, is the safe habit.

3.2 Your `info.yaml`

Tiny Tapeout reads this file to learn about your project. The important entries:

Entry	Meaning
`title`, `author`, `description`	Shown on the Tiny Tapeout website and in the printed datasheet.
`language: "SystemVerilog"`	Tells the tools how to read your sources.
`clock_hz: 50000000`	The clock you want on the real chip (50 MHz). The demo board generates it.
`tiles: "6x4"`	How much of the shared chip you use: 6 tiles wide, 4 high. This picks the die outline, 1289.28 × 710.64 µm.
`top_module`	The module the flow builds. Must start with `tt_um_` and be unique; yours is `tt_um_vertrex_star_proem`.
`source_files`	Every `.sv` file, relative to `src/`. A file missing here is invisible to the flow.
`pinout`	Names for each pin, used only for the datasheet and website. It does not change the hardware.
`yaml_version: 6`	The format version; never change it.

When you add a new SystemVerilog file, add it in **three** places: `info.yaml` (`source_files`), `test/Makefile` (`PROJECT_SOURCES`), and `src/files.f` (used by the local scripts).

3.3 Workflow files are YAML too

The files in `.github/workflows/` are also YAML. They are explained in Section 12.

4 JSON and `src/config.json`

JSON is YAML's stricter cousin: maps are written `{ "name": value }`, lists `[a, b]`, and every name is in double quotes. JSON has no comments, so the Tiny Tapeout template uses a trick: entries called `"//"` hold explanatory text, and the tools ignore them.

`src/config.json` configures LibreLane, the ASIC flow. The settings that matter to you:

Setting	Meaning
`CLOCK_PERIOD: 20`	The clock period in nanoseconds the flow must meet (20 ns = 50 MHz).
`PL_TARGET_DENSITY_PCT: 60`	How tightly cells are packed during placement. Higher packs cells closer (shorter wires) but makes routing harder.
`PL_RESIZER_HOLD_SLACK_MARGIN`, `GRT_RESIZER_HOLD_SLACK_MARGIN`	Extra safety the tool adds when fixing hold time (Section 9).
`MAX_TRANSITION_CONSTRAINT: 0.75`	Added for ProEm: no signal may take longer than 0.75 ns to switch. This made the slow corner pass.
`PL_RESIZER_SETUP_SLACK_MARGIN: 1.0`, `GRT_RESIZER_SETUP_SLACK_MARGIN: 0.5`	Added for ProEm: the resizer aims for 1.0 ns and 0.5 ns of spare setup time rather than just enough.
`FP_PDN_VPITCH`, `FP_PDN_VWIDTH`	Spacing and width of the power stripes; set by the CMOS5L template.

Everything below the template's "DO NOT CHANGE ANYTHING BELOW THIS POINT" line should be left alone: it matches the Tiny Tapeout chip.

5 Simulation

5.1 What a simulator does

A simulator runs your Verilog on a normal computer so you can check its behaviour before building anything. It keeps a list of events ("clock rises at 20 ns", "this signal changes") and processes them in time order. You met this in ModelSim. The open-source world has several simulators; ProEm uses three.

Tool	Role in ProEm
Icarus Verilog	Runs the cocotb test, on your RTL and on the gate-level netlist. Used by Tiny Tapeout's GitHub Actions.
Verilator	Used by the flow as a **linter**: it reads your code and reports mistakes such as width mismatches, without simulating.
Yosys CXXRTL	Turns your design into C++ so the 13 pin-level tests in `sim/` can run fast.

5.2 Icarus Verilog

Icarus works in two steps, like a compiler:

```
iverilog -g2012 -o sim.vvp tb.v src/...sv   # compile (-g2012 = SystemVerilog 2012)
vvp sim.vvp                                  # run
```

`iverilog` checks and compiles your files into `sim.vvp`; `vvp` runs it. You rarely type these yourself because cocotb's Makefile does it for you. Icarus supports most, but not all, SystemVerilog; ProEm's code style (`logic`, `always_ff`, `always_comb`) is well supported. Tiny Tapeout's Actions use Icarus version 13; Ubuntu's package is version 12, which is fine for RTL tests.

5.3 Waveforms

The test writes a waveform file, `test/tb.fst`, which records every signal over time, like ModelSim's wave window. Open it with **GTKWave** or **Surfer** (both free). FST is a compressed format; VCD is the older, larger equivalent.

5.4 RTL and gate-level simulation

- **RTL simulation** runs the SystemVerilog you wrote. Fast, easy to debug.
- **Gate-level (GL) simulation** runs the **netlist**: the list of real standard cells (AND gates, flip-flops) the flow produced. Each cell comes with a Verilog model from the PDK (`sg13cmos5l_stdcell.v`). It is slower, but it proves the flow did not change your design's behaviour, and it catches things RTL simulation hides, such as a register without a reset starting as `X`.

ProEm's 5 cocotb tests pass in both modes on CMOS5L.

5.5 Your C++ test suite (CXXRTL)

`scripts/run.sh` asks Yosys to translate the design to C++ (CXXRTL), then compiles each test in `sim/` against it. `sim/bench.h` holds the shared helpers: it sends instruction bytes over the RX pin at a chosen baud rate, decodes what comes back on TX, and lets each test model a real device on the pins (an SPI slave, an I^2C controller, a USB host). These are your deepest tests: 13 standards and scenarios plus 1,800 random programs and 600 random branch programs. The `sim` workflow runs them on every push.

6 cocotb

6.1 What cocotb is

cocotb ("coroutine co-simulation testbench") lets you write the testbench in **Python** instead of Verilog. The simulator (Icarus) runs the hardware; cocotb runs your Python alongside it and can read and drive any signal. Python makes it easy to build instruction bytes, keep lists of expected results, and print clear messages.

6.2 The pieces in `test/`

File	Role
`tb.v`	A small Verilog top level that instantiates your chip, adds wiring the test needs (for example, looping channel B's output back to channel A, and an open-drain I^2C bus), and starts the waveform dump.
`test.py`	The tests themselves. Each function marked `@cocotb.test()` is one test.
`Makefile`	Lists the source files and includes cocotb's own make rules, which call Icarus. `GATES=yes` switches to the gate-level netlist.
`requirements.txt`	The Python packages to install (`cocotb==2.0.1`, `pytest`).

6.3 Reading a test

A shortened version of `test_uart_loopback`:

```
@cocotb.test()
async def test_uart_loopback(dut):
    link = await power_on(dut)          # start clock, reset, send 0x55
    dut.loop_uart.value = 1             # wire B's output into A's input
    ...
    await link.send(program + link.run(0))    # program the chip
    await ClockCycles(dut.clk, 3 * 10 * r + 200)  # let it run
    assert link.got[0] == expected, f"channel A read {link.got[0]}"
```

- `dut` ("device under test") is a handle to `tb.v`; `dut.ui_in.value = 1` drives a pin, `dut.uo_out.value` reads one.
- `async` and `await` are how cocotb waits for simulated time. `await ClockCycles(dut.clk, 100)` pauses the Python until 100 clock edges have passed.
- `assert` checks a result; if it is false the test fails and prints the message.
- The `Link` class in `test.py` is your helper: it turns calls such as `link.main(9, 0)` into instruction bytes, sends them bit by bit on `ui_in[0]`, and decodes the chip's replies from `uo_out[0]`.

6.4 Running it yourself (Windows)

Install WSL (Windows Subsystem for Linux) from the Microsoft Store ("Ubuntu"), open it, and run:

```
sudo apt update && sudo apt install -y iverilog python3-pip python3-venv git make
python3 -m venv ~/cocotb && source ~/cocotb/bin/activate
cd /mnt/c/path/to/ProEm/test      # your Windows folder, seen from Linux
pip install -r requirements.txt
make -B                           # runs the 5 tests on the RTL
```

The last lines of the output say `TESTS=5 PASS=5 FAIL=0`. Results are also written to `results.xml`, which is what the GitHub `test` workflow checks. Section 7 goes further: running one test, waveforms, writing your own test, and the C++ suite.

7 Hands-on: simulating ProEm yourself

This section is a practical walkthrough. It shows you how to run both kinds of test on your own computer, read the results, and write a new test of your own. Every command and example here was run on the current repository and passes.

7.1 Which one to use

	cocotb (Python, Icarus)	C++ (CXXRTL)
Speed	Slow: seconds to minutes per test	Fast: thousands of programs in a minute
Waveforms	Yes, `test/tb.fst`	Not set up
Gate-level netlist	Yes (`GATES=yes`)	No, RTL only
Run by GitHub	`test` and `gl_test` jobs	`sim` workflow
Best for	Looking at signals, debugging one scenario, checking the real netlist	Many scenarios, random testing, quick checks after every change

A good habit: change the RTL, run the C++ suite (fast), and when something fails and you need to see why, reproduce it in cocotb and look at the waveform.

7.2 One-time setup (Windows, with WSL)

Open the Ubuntu (WSL) terminal and install everything both kinds of test need:

```
sudo apt update
sudo apt install -y iverilog g++ make git python3-pip python3-venv gtkwave
python3 -m venv ~/proem-env
source ~/proem-env/bin/activate        # run this line again in every new terminal
pip install cocotb==2.0.1 pytest yowasp-yosys
```

`yowasp-yosys` is Yosys packaged for Python, so you do not need to build it. Your Windows folders appear under `/mnt/c/`, for example `/mnt/c/Users/Paul/Documents/ProEm`.

7.3 cocotb, step by step

Run every test (RTL). From the `test` folder:

```
cd /mnt/c/Users/Paul/Documents/ProEm/test
make -B
```

`-B` forces a rebuild, so your latest RTL is always used. Each test prints `running` and then `passed` or `failed`, and the summary line reads `TESTS=5 PASS=5 FAIL=0`.

Run one test only. Name it with `COCOTB_TEST_FILTER`:

```
make -B COCOTB_TEST_FILTER=test_spi_loopback
```

Look at the waveform. Every run writes `test/tb.fst`:

```
gtkwave tb.fst
```

In GTKWave, expand `tb` and then `user_project` in the left panel to reach your design's hierarchy (`core`, `channel_a`, and so on). Drag signals into the wave view, and use the zoom buttons to fit. `uo_out[0]` is the chip's TX line, `ui_in[0]` is RX, and `uio_out` and `uio_oe` are the protocol pins.

Write a new test. Add this to the end of `test/test.py`. It sends one UART byte (0xA5) into pin 0 from Python and checks that channel A reports it:

```
@cocotb.test()
async def test_my_uart_byte(dut):
    link = await power_on(dut)          # clock, reset, auto-baud
    r = 16                              # 16 clocks per UART bit
    link.channel = 0
    await link.send(link.line(0, 0)     # plain line coding on pin 0
                    + link.clock(ASYNC, r)
                    + link.start(0x00, 1, 0, 0, 0)   # wait for the start bit
                    + link.main(9, 0)   # read 9 bits: 8 data + stop
                    + link.run(0))
    byte = 0xA5
    for level in [0] + [(byte >> i) & 1 for i in range(8)] + [1]:
        dut.ext_in.value = 0xFE | level # pin 0 carries our UART line
        await ClockCycles(dut.clk, r)
    await link.quiet()                  # wait until the chip stops replying
    dut._log.info(f"channel A read {link.got[0]}, status {link.status[0]}")
    assert link.got[0] == [(8, 0xA5), (1, 1)], f"got {link.got[0]}"
    assert link.status[0] == [1], f"status {link.status[0]}"
```

Run it with `make -B COCOTB_TEST_FILTER=test_my_uart_byte`. The log shows `channel A read [(8, 165), (1, 1)], status [1]`: 8 bits with value 165 (0xA5), then the stop bit, then status 0x01 (frame complete).

How the pieces fit:

- `power_on` (in `test.py`) starts a 50 MHz clock, resets the chip, sends 0x55 for auto-baud, and starts a background monitor that decodes everything the chip sends back into `link.got` (data, per channel, as (bits, value) pairs) and `link.status` (one status byte per frame).
- `dut.ext_in` is a register in `tb.v` that drives the protocol pins from outside, like a device connected to the chip. Bit 0 is pin 0.
- `link.line`, `link.clock`, `link.start`, `link.main` and `link.run` build the same instruction bytes the ISA document describes.
- `dut._log.info` prints to the log; use it freely while debugging.

Reading internal signals. Any signal in the design can be read from Python: for example `dut.user_project.core.channel_a.chain.base_mode.value`. Reading a signal that is `X` (unknown) and converting it with `int(...)` raises "Can't convert LogicArray to int". That usually means a register without a reset, or a signal read before reset finished.

Gate-level simulation on your computer. This is optional, because GitHub's `gl_test` job already does it on every push. To run it yourself:

1. From a green `gds` run, download the `tt_submission` artifact. Inside it, `tt_um_vertrex_star_proem.v` is the routed netlist; copy it into `test/` and rename it `gate_level_netlist.v`.
2. Install the CMOS5L PDK at the revision the CI uses:

```
mkdir ~/pdk && cd ~/pdk && git init -q
git fetch --depth 1 https://github.com/IHP-GmbH/IHP-Open-PDK.git 2
    bbec755dc67ca3db0261c3d6163e15735d66710
git checkout -q FETCH_HEAD
```

3. Install Tiny Tapeout's Icarus 13. Ubuntu's Icarus 12 produces `X` values with the CMOS5L cell models, so the test fails even though the design is fine:

```
wget https://github.com/TinyTapeout/iverilog/releases/download/v13.0/iverilog_13
    .0-1_amd64.deb
sudo apt install -y ./iverilog_13.0-1_amd64.deb
```

4. Run it: `cd test && make -B GATES=yes PDK_ROOT=$HOME/pdk`. This takes several minutes, because every one of the 20,000 cells is simulated.

7.4 C++ (CXXRTL), step by step

What happens. `scripts/run.sh` does three things:

1. Yosys reads every file listed in `src/files.f` and writes the whole chip as one C++ class into `build/emu_sim.h`. Each port becomes a member: `rx_serial` becomes `p_rx__serial` (`p_` in front, and each underscore doubled).
2. `g++` compiles each test in `sim/` together with that header, producing programs such as `build/t_ps2`.
3. It runs every test with the settings it needs, then (unless `SKIP_SYNTH=1`) synthesises the design to report its size.

Run everything. From the repository folder:

```
cd /mnt/c/Users/Paul/Documents/ProEm
SKIP_SYNTH=1 bash scripts/run.sh      # all tests, about a minute
bash scripts/run.sh                    # tests, then synthesis area
```

Each check prints a line ending in `PASS` or `FAIL` with what was received, and each program ends with `ALL PASS` or `FAILED`. The script stops at the first failure.

Run one test again. After one full run, the programs are in `build/`. The number after the name is the setting the test takes; `scripts/run.sh` shows the ones it uses:

```
./build/t_ps2 40          # PS/2 with a 40-clock half period
./build/t_spi 6           # SPI with SCK half period 6 clocks
./build/t_random 3 1 150  # seed 3, MSB first, 150 random programs
```

Rebuild after changing the RTL. Run `scripts/run.sh` again. It regenerates `emu_sim.h` and recompiles everything, which is the simplest way to be sure nothing is stale.

The helpers in `sim/bench.h`. Every test includes this file. The functions you use most:

Helper	What it does
`power_on()`	Resets the chip and sends 0x55 for auto-baud.
`CH = 0` or `1`	Which channel the next instructions address.
`I_line`, `I_clock`, `I_start`, `I_hdr`, `I_main`, `I_end`, `I_data`, `I_branch`, `I_run`, and the rest	Build one instruction's bytes, following the ISA.
`cat({...})`	Joins several instructions into one list.
`program(bytes)`	Sends the bytes to the chip over RX and waits until they are delivered.
`idle(n)`	Runs `n` clock cycles.
`env` and `pin_in`	`env` is a function called every clock cycle; set `pin_in` inside it to drive the protocol pins, as a connected device would.
`t.p_data__out__a.get<bool>()`	Reads a chip output (here channel A's data pin value).
`wait_return()`	Runs until the chip has finished sending results.
`got[c]`, `status[c]`	What channel `c` sent back: data as (bits, value) pairs, and one status byte per frame.
`report(name, ok, detail)`	Prints one `PASS` or `FAIL` line.

Write a new test. The same UART check as the cocotb example, in C++. Save it as `sim/t_my_uart.cpp`:

```
#include "bench.h"
int line_level = 1;
void my_env() { pin_in = 0xFE | line_level; }   // pin 0 is our UART line

void uart_send(unsigned b, int r) {
  int bits[10]; bits[0] = 0;                     // start bit
  for (int i = 0; i < 8; i++) bits[1 + i] = (b >> i) & 1;
  bits[9] = 1;                                   // stop bit
  for (int i = 0; i < 10; i++) { line_level = bits[i]; idle(r); }
}

int main() {
  env = my_env; power_on();
  int r = 16; CH = 0;
  program(cat({I_line(0, 0), I_clock(ASYNC, r), I_start(0, 1, 0, 0, 0),
               I_main(9, 0), I_run(0)}));
  uart_send(0xA5, r);
  wait_return();
  V exp = {{8, 0xA5}, {1, 1}};
  bool ok = report("UART receive of one byte",
                   got[0] == exp && status[0] == std::vector<unsigned>({1}),
                   show(got[0]) + " |" + showst(status[0]));
  printf(ok ? "ALL PASS\n" : "FAILED\n");
  return !ok;
}
```

To compile and run it by itself (after `scripts/run.sh` has created `build/emu_sim.h` once):

```
RT=$(python3 -c "import yowasp_yosys,os;print(os.path.join(os.path.dirname(yowasp_yosys.
    __file__),'share/include/backends/cxxrtl/runtime'))")
g++ -std=c++14 -O2 -I"$RT" -Ibuild sim/t_my_uart.cpp -o build/t_my_uart
./build/t_my_uart
```

It prints `UART receive of one byte ...  8:A5 1:01 | 01 PASS` and `ALL PASS`. To make it part of the suite, add `t_my_uart` to the `TESTS` list in `scripts/run.sh` and add a line `./build/t_my_uart`

with the others.

Random tests and seeds. `t_random` and `t_random_branch` generate random programs from a **seed** (a starting number for the random generator). The same seed always gives the same programs, so when one fails, rerun it with that seed to reproduce the failure exactly, then rebuild the failing program in cocotb if you need waveforms.

Common problems.

- `yosys: command not found`: activate the environment: `source ~/proem-env/bin/activate`.
- `emu_sim.h: No such file`: run `scripts/run.sh` once before compiling a test by hand.
- A test hangs: it is waiting for a reply that never comes. Check the program (wrong channel in `CH`, missing `I_run`) and that `env` drives the pins the test expects.
- A new source file is not seen: add it to `src/files.f` (and to `info.yaml` and `test/Makefile`).

8 The ASIC flow (LibreLane)

LibreLane (the successor of OpenLane) is a script that runs about 80 steps in order, calling several open-source programs. Each step's output is in `runs/wokwi/NN-step-name/`. The main stages, with ProEm's numbers from the CMOS5L build:

8.1 1. Lint and synthesis (Verilator, Yosys)

Verilator checks the code. Then **Yosys** performs **synthesis**: it turns your RTL into a netlist of **standard cells**, the pre-designed gates the factory offers (for example `sg13cmos5l_nand2_1`: a 2-input NAND, drive strength 1). This is what Quartus's "Analysis and Synthesis" does for LUTs; here the building blocks are real gates. ProEm becomes about 15,500 cells.

8.2 2. Floorplan and power grid

The flow creates the empty die (6×4 tiles), places the pins where Tiny Tapeout expects them, draws rows for cells to sit in, and lays a **power distribution network** (PDN): metal stripes carrying the supply (VPWR, 1.2 V) and ground (VGND) to every row. These are the regular vertical stripes in the layout pictures.

8.3 3. Placement (OpenROAD)

Every cell gets a position. Global placement spreads cells to keep connected ones close; detailed placement snaps them into legal row positions without overlap.

8.4 4. Clock tree synthesis (CTS)

Your 2,400 flip-flops all need the clock at nearly the same moment. CTS builds a tree of buffers (about 300 cells) so the clock arrives everywhere with little **skew** (difference in arrival time).

8.5 5. Resizing and timing repair

The **resizer** swaps cells for stronger or weaker versions and inserts buffers to fix timing. On ProEm it adds about 3,300 delay cells (`dlygate4sd3`) purely to fix **hold** time. This is normal and is why the cell count grows from about 15,500 to about 19,700.

8.6 6. Routing

Global routing plans which region each wire passes through; **detailed routing** draws every wire exactly, on the metal layers, obeying the factory's spacing rules. ProEm's wires total about 850 mm. The router iterates until there are 0 violations.

8.7 7. Antenna repair, fill and extraction

- **Antennas**: during manufacturing, a long metal wire connected only to a transistor gate can collect charge and damage it. The flow adds small diodes where needed.
- **Fill**: empty space is filled with filler and decoupling cells so the chip is uniform to manufacture. That is why the lower part of your layout is not empty.
- **RC extraction**: the flow calculates the real resistance and capacitance of every wire (the `.spef` file) so the final timing check uses real wire delays.

8.8 8. Sign-off

Check	What it proves
STA (static timing analysis)	Every path meets setup and hold at every corner (Section 9).
DRC (design rule check), Magic and KLayout	The drawing obeys every manufacturing rule (minimum widths, spacings, overlaps). A single DRC error can make the chip fail.
LVS (layout versus schematic), Netgen	The transistors actually drawn connect exactly as the netlist says.
IR drop	The power grid delivers the supply without a large voltage loss (ProEm: under 1 mV).

8.9 9. Output

The final GDS (`tt_um_vertrex_star_proem.gds`) is the drawing sent to the factory. The **LEF** is a simplified outline with only pin positions, which Tiny Tapeout uses to fit your block into the full chip.

9 Timing, explained simply

9.1 Setup and hold

Between two flip-flops sits logic. When the clock ticks, the first flip-flop launches a new value; it travels through the logic and must reach the second flip-flop:

- **Setup**: early enough before the *next* clock edge. If the logic is too slow, the second flip-flop captures the old value. A setup failure can be fixed by slowing the clock.
- **Hold**: not so early that it overwrites the value being captured at the *same* edge. A hold failure cannot be fixed by slowing the clock, which is why the flow adds delay cells for it.

Slack is the spare time. Positive slack means the path passes; negative slack is a violation. The **maximum clock** is roughly 1/(period − setup slack): ProEm's slowest path takes $20 - 2.6 = 17.4$ ns at the slow corner, so about 57 MHz.

9.2 Corners

Real chips vary. Transistors are slower when hot, at low voltage, or when the factory makes them slightly "slow". Tools therefore check **corners** (process, voltage, temperature):

Corner	Supply	Temperature	ProEm setup slack	Hold slack
Slow	1.08 V	125 °C	2.6 ns	0.61 ns
Typical	1.20 V	25 °C	8.9 ns	0.28 ns
Fast	1.32 V	−40 °C	12.5 ns	0.11 ns

Setup is hardest at the slow corner; hold is hardest at the fast corner. The CMOS5L flow only *enforces* the typical corner, so a build can be green while failing the slow one. That happened to ProEm at first (the slow corner missed by 1.2 ns); the extra settings in `config.json` fixed it.

9.3 Slew

Slew (or transition time) is how long a signal takes to switch from 0 to 1. A small cell driving a long wire switches slowly, and a slow edge also slows every gate after it. ProEm's failing path had edges over 2 ns long; limiting slew to 0.75 ns forced the tool to use stronger cells and buffers.

10 Processes and PDKs

10.1 What a process is

A **process** is a factory's recipe for making chips: the transistor size (here 130 nm, the length of the smallest transistor gate), the number of metal layers for wiring, and the supply voltages. A **PDK** (process design kit) is the package the factory gives designers: the rules (for DRC), the standard cells with their layouts and timing models (`.lib` files), and the Verilog models used in gate-level simulation. IHP, SkyWater and GlobalFoundries publish open PDKs, which is what makes Tiny Tapeout possible.

10.2 The processes you have met

Process	Notes
IHP SG13CMOS5L	**Your target for Jane Street.** 130 nm CMOS from IHP (Germany). Five routing layers: Metal1 to Metal4 plus one thick TopMetal1. Core supply 1.2 V, I/O 3.3 V. Cell names start `sg13cmos5l_`.
IHP SG13G2	The full IHP 130 nm process, with seven metal layers and fast bipolar transistors (for radio circuits). Your earlier reference build used it; same standard cells, more routing layers.
SkyWater SKY130	The first open PDK, used by earlier Tiny Tapeout shuttles and by the ASIC you reverse engineered.
GF180	GlobalFoundries 180 nm; another Tiny Tapeout option.

Fewer metal layers means fewer "floors" for wires, so routing is harder and wires are longer. That is one reason CMOS5L timing is a little slower than SG13G2 for the same design.

11 How Tiny Tapeout works

11.1 Sharing one chip

Making a chip costs far too much for one small design, so **Tiny Tapeout** puts hundreds of projects on one die. The die is divided into **tiles**; each project gets a rectangle of tiles (yours: 6 × 4). A batch of designs manufactured together is a **shuttle**. After manufacturing (several months), chips

are mounted on a **demo board**. A small microcontroller on the board picks one project at a time through an on-chip multiplexer, supplies its clock and reset, and connects its pins to the board's headers.

11.2 Your pins

Every project has the same interface, which is why your top module has exactly these ports:

Port	Meaning
`ui_in[7:0]`	8 inputs. ProEm uses `ui_in[0]` as RX from the host.
`uo_out[7:0]`	8 outputs. ProEm: TX, BAUD_LOCKED, ARMED, HOST_FULL, SELECT.
`uio_in`, `uio_out`, `uio_oe`	8 bidirectional pins. `uio_oe[i] = 1` makes pin i an output. ProEm's protocol pins are here, so every line can be driven and read back.
`clk`, `rst_n`	Clock, and reset (active low: 0 means reset).
`ena`	1 when your project is the one selected.

11.3 Templates

You start from Tiny Tapeout's template repository. It contains the workflows, `src/config.json`, a sample `info.yaml`, and the cocotb test layout. For Jane Street, the template is the `cmos5l` branch of Tiny Tapeout's IHP template; ProEm now matches it.

12 GitHub Actions: what happens when you push

12.1 Workflows

A **workflow** is a YAML file in `.github/workflows/`. The line `on: push` means "run this every time someone pushes". A workflow has **jobs**; each job runs on a fresh Linux machine (`runs-on: ubuntu-24.04`) and has **steps**. A step either runs a command (`run:`) or uses a ready-made action (`uses:`). `needs: gds` means "wait for the `gds` job to finish first".

```
jobs:
  gds:
    runs-on: ubuntu-24.04
    steps:
      - name: Build GDS
        uses: TinyTapeout/tt-gds-action@ihp-cmos5l   # Tiny Tapeout's action, CMOS5L
    version
        with:
          pdk: ihp-sg13cmos5l
  precheck:
    needs: gds
    ...
```

12.2 ProEm's workflows

Workflow and job	What it does
`gds` / `gds`	Installs the CMOS5L PDK and LibreLane, runs the whole ASIC flow, and uploads the results. **This is the one Jane Street cares about.**
`gds` / `precheck`	Tiny Tapeout's own checks on your GDS: full KLayout DRC, pins in the right places, right outline, no forbidden layers.
`gds` / `gl_test`	Runs the cocotb test on the gate-level netlist the `gds` job produced.
`gds` / `viewer`	Publishes a 3D viewer of your layout to GitHub Pages. On a private repository it fails (Pages needs a paid plan), so it is marked `continue-on-error`: its failure does not turn the run red. It is not needed for submission.
`test`	The cocotb test on the RTL.
`sim`	The C++ test suite.
`docs`	Checks `docs/info.md` (your datasheet page) can be built.
`formal`	Runs every formal proof (`scripts/formal.sh`) with the OSS CAD Suite.

12.3 Reading the results

On GitHub, open the **Actions** tab. Each push is a row; a green tick means every job passed. Click a run to see each job; click a job to see each step's log; the `gds` job's summary shows the cell counts, routing statistics and linter output. At the bottom of the run page, **Artifacts** are downloadable zips:

- `tt_submission`: the GDS, LEF, netlist and settings, the files that go onto the chip.
- `GDS_logs`: every step's reports, for debugging.
- `gds_render`: a picture of the layout.

If a job fails, open it and read the last red lines of the failing step. Common causes: a file missing from `info.yaml`, a Verilog error the linter reports, or a timing violation at the typical corner.

A private repository gets a monthly allowance of free Actions minutes (on GitHub's free plan, 2,000 minutes). A full `gds` run takes tens of minutes, and the precheck DRC can take much longer, so avoid pushing many times in a row just to test small edits.

13 Git in five commands

Command	Meaning
`git status`	What has changed since the last commit.
`git add file`	Choose a change to include in the next commit. `git add -A` includes everything.
`git commit -m "message"`	Save a snapshot with a description.
`git push`	Send your commits to GitHub; this triggers the Actions.
`git log -oneline`	List the commits.

A **commit** is a snapshot of the whole project. A **remote** (`origin`) is the copy on GitHub. "Everything up to date" after `git push` means there were no new commits to send: you forgot `git add` and `git commit`. "Not a git repository" means your terminal is not in the folder that contains the hidden `.git` folder.

14 What to do next

1. Push the latest repository (CMOS5L template) to GitHub.

2. Open the Actions tab and wait for `gds`, `precheck` and `gl_test` to go green. Download the `tt_submission` artifact and keep it.

3. Install WSL and run both test suites yourself once (Section 7), so you can change and rerun tests without waiting for GitHub.

4. Open `test/tb.fst` in GTKWave or Surfer and follow one UART byte through the design.

5. Open the GDS in KLayout (instructions in `gds/README.md`) and find the flip-flops, the power stripes and the pins.

6. Read the design and ISA documents, and the formal verification guide; run `scripts/formal.sh` once the OSS CAD Suite is installed.

7. When Jane Street update their blog post, check the submission steps against it.

15 Glossary

ASIC
Application-specific integrated circuit: a chip made for one design, unlike an FPGA.

cocotb
Python testbench library that drives a Verilog simulator.

Corner
A combination of process, voltage and temperature at which timing is checked.

CTS
Clock tree synthesis: building the buffer tree that distributes the clock.

CXXRTL
Yosys feature that turns Verilog into C++ for fast simulation.

DRC
Design rule check: the layout obeys the factory's geometric rules.

Fill cells
Cells with no logic, placed in empty space for manufacturability.

GDS
The file format of the chip layout sent to the factory.

Gate level
Simulation of the synthesised netlist of real cells rather than the RTL.

GitHub Actions
GitHub's machines that run your workflows on every push.

Hold
Timing check that data does not change too soon after a clock edge.

Icarus Verilog
Open-source Verilog simulator (`iverilog`, `vvp`).

LEF
Simplified outline of a block with its pin positions.

LibreLane
The open-source flow that runs synthesis to GDS.

Lint
Automatic check of code for likely mistakes without simulating.

LVS
Layout versus schematic: the drawn circuit matches the netlist.

Netlist
A design described as connected standard cells.

OpenROAD
The program that does floorplanning, placement, CTS, routing and timing.

PDK
Process design kit: rules, cells and models for a manufacturing process.

PDN
Power distribution network: the supply and ground wiring.

Precheck
Tiny Tapeout's final checks before a design is accepted.

RTL
Register-transfer level: the SystemVerilog you write.

Setup
Timing check that data arrives early enough before the clock edge.

Shuttle
A batch of designs manufactured together.

Slack
Spare time on a timing path; negative means a violation.

Slew
How long a signal takes to switch.

SPEF
File of extracted wire resistances and capacitances.

Standard cell
A pre-designed gate or flip-flop from the PDK.

STA
Static timing analysis: checking all paths' timing without simulation.

Synthesis
Turning RTL into a netlist of standard cells (Yosys).

Tile
The unit of area on a Tiny Tapeout chip (about $202 \times 155\,\mu\text{m}$ on IHP).

Utilisation
Share of the core area covered by cells (ProEm: 35 %).

Verilator
Open-source Verilog linter and fast simulator.

Workflow
A YAML recipe that GitHub Actions runs.

YAML
Human-readable settings format based on indentation.

Yosys
Open-source synthesis tool.